Week 8: Greedy reasoning and mixed assessment

Decisions from a single scan

When each position allows a different travel distance, determine reachability and the fewest moves. On a circular route, find a starting point that never runs out of fuel.

Solving these tasks without storing every choice requires a reason that discarded candidates cannot be needed. Reachable ranges and prefix sums provide that reason.

Week 8 learning outcomes

Sessions 15 and 16: Reachable ranges, candidate elimination, counterexamples, and mixed assessment

- Check contracts and boundary conditions.
- Trace state and implement in Python.
- Explain correctness, time, auxiliary space, and output space.

Opening scenarios, traces, and checks are additional teaching examples.

Session 15 · Reachability frontiers and candidate elimination

Determine reachability, minimum jumps, and a circular-route start. Greedy choices require proof that discarded alternatives cannot improve the answer.

Prerequisites and Python semantics

range(n-1) excludes the last index. enumerate supplies positions and values; zip pairs corresponding entries but stops at the shorter input, so check the equal-length contract.

Minimum-jump contract

- Each value: maximum forward jump length
- Every distance up to that value is allowed
- Official problem 45 guarantees reachability
- Do not count another jump at the last index

To support unreachable inputs, check `can_jump` first or define an explicit failure return.

55 · Problem and state

LeetCode 55: Jump Game · Medium

Decide whether maximum forward jump lengths permit reaching the last index.

`nums=[2,0,1,0] → True`

Update farthest only while index is reachable.

55 · Implementation

```
def can_jump(nums):
    farthest = 0
    for index, step in enumerate(nums):
        if index > farthest:
            return False
        farthest = max(farthest, index + step)
    return True
```

55 · Correctness and cost

Reachable positions form the interval 0..farthest. Allowed jumps include every shorter distance, preserving an interval; an index beyond it cannot be crossed.

n is length. O(n) time,O(1) space.

45 · Problem and state

LeetCode 45: Jump Game II · Medium

Return the minimum jumps needed to reach the last index.

`nums=[2,3,0,1,0] → 2`

end is the current frontier, farthest the next; increment jumps only at index==end.

45 · Implementation

```
def min_jumps(nums):
    end = farthest = jumps = 0
    for index in range(len(nums) - 1):
        farthest = max(farthest, index + nums[index])
        if index == end:
            jumps += 1
            end = farthest
    return jumps
```

45 · Correctness and cost

Scan every reachable position in the current level before fixing the next frontier. These are BFS levels in an implicit unweighted graph, so the jump count is minimal.

n is length. O(n) time,O(1) auxiliary space.

134 · Problem and state

LeetCode 134: Gas Station · Medium

Return a start completing a circular route from an empty tank, or -1.

`gas=[1,4,2], cost=[3,1,3] → 1`

Accumulate total difference and candidate tank; restart at i+1 after tank becomes negative.

134 · Implementation

```
def gas_start(gas, cost):
    total = tank = start = 0
    for index, (supply, use) in enumerate(zip(gas, cost)):
        total += supply - use
        tank += supply - use
        if tank < 0:
            start, tank = index + 1, 0
    return start if total >= 0 else -1
```

134 · Correctness and cost

At the first failure over s..i, each earlier prefix s..j-1 is nonnegative. Removing such a prefix leaves j..i negative too, eliminating all intermediate starts. If total is nonnegative, the final candidate also has enough surplus to wrap through the eliminated prefixes.

n is station count. O(n) time,O(1) space.

State trace

i (nums=2,3,0,1,0)	farthest	end	jumps
0	2	2	1
1	4	2	1
2	4	4	2
3	4	4	2

State is shown after each index update; only a frontier boundary increments jumps.

Failure inputs and concept checks

In [2,3,0,0,0], taking the longest first jump stops at the zero at index 2. Exploring reachable index 1 also exposes a path to the last index. The algorithm maintains the range reachable from all eligible positions.

Does one successful example prove a greedy rule?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Session 16 · Mixed assessment without pattern labels

Start from inputs, outputs, and constraints without a topic label. Observe familiar-problem fluency separately from transfer to changed conditions.

Prerequisites and Python semantics

Check equality versus identity, mutation policy, and return types. A correct value can still violate a no-mutation contract.

200 · Problem and state

LeetCode 200: Number of Islands · Medium

Count land components connected horizontally or vertically in a character grid.

`grid=[["1","0","1"],["1","0","0"]] → 2`

Start a stack traversal at each unseen land cell and mark its whole component.

200 · Implementation

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

200 · Correctness and cost

A traversal reaches exactly its starting component. Marked components cannot restart; the number of starts equals the component count.

R,C are dimensions. O(RC) time and space; general adjacency lists take O(V+E) time.

State trace

assessment phase	learner record	instructor evidence
definition	input/output/constraints	added assumptions
design	state/elimination	reasoning consistency
execution	small/boundary inputs	first failed state
explanation	time/space	copy/output/stack costs

Use the records to locate the first incorrect assumption or state.

Failure inputs and concept checks

Use == for values; is tests identity. Diagonal land in a 2x2 grid forms two components.

Does success on a repeated problem establish transfer?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Eliminating starts in a failed segment

i	gas[i]−cost[i]	Candidate tank	Next start
0	-2	-2 → 0	1
1	3	3	1
2	-1	2	1

The total is 0. Starting at index 1 completes the circuit. Gas and cost have equal lengths; tank capacity is unlimited.

Checkpoint questions

1. Is the last index reachable in `[2,0,0,1]`?
2. Find the minimum jumps for `[2,3,0,1,0]`.
3. Find a start for gas=`[1,4,2]`, cost=`[3,1,3]`.
4. How many jumps are needed for a reachable length-one array?

Independent practice and connections

Student exercises

Hints and solutions

Instructor guide

Download Week 8 code

Submit code, state definition, correctness argument, complexity, and boundary inputs. Week 8 includes a mixed assessment without pattern labels.

Mixed assessment

Open the mixed assessment for one familiar problem and a job-selection task with changed conditions.

- Record input, output, and constraints first.
- Submit method selection, reasoning, and boundary inputs.
- Compare with the instructor key after submission.

The job-selection task is an additional teaching problem.

Execution and sources

Run in the `code` folder of the weekly code archive.

```
python demo_15.py
python demo_16.py
python verify.py
python verify.py --student
```

Use the last command after implementing the student file.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).